

LIVE VIDEO STREAMER VIA NETWORK

FEATURE: INTERNET MODE

Version 2.0 Technical Specification

Cross-Network & Zero-Configuration LAN Video Streaming

Complete architectural, wire protocol, and implementation specification for high-throughput, low-latency live camera streaming over local networks and the public internet.

Author: Aadarsh Sinha and Krishnashis Das
GitHub Repository: [Live-video-streamer-via-network](#)

Document Revision: 2.0

System Specification & Developer Guide

This document is an authoritative engineering specification. All client and server implementations must strictly conform to the 22-byte header framing and network rules defined herein.

Document Control

Field	Value
Document Title	Live Video Streamer via Network (Internet Mode) Specification
Document ID	DOC-LVS-2026-V2
Version	2.0
Status	Release Candidate / Final
Author(s)	Aadarsh Sinha and Krishnashis Das
Organization	Live Video Streamer Project
Release Date	August 26, 2026
Last Updated	August 26, 2026
Distribution	Open Source / Technical Public Specification
Classification	Technical Engineering Document

Security

Protocol Version Compatibility Notice: Version 2.0 introduces a 22-byte binary header with a `uint32` payload length field (`MAX_PAYLOAD = 8 MB`). Version 1.0 clients (20-byte header with `uint16` payload length) are incompatible with Version 2.0 endpoints by design. Senders and receivers across the network must use identical protocol revisions to ensure clean packet parsing.

Revision History

Version	Date	Author(s)	Changes
1.0	2026-08-01	Aadarsh Sinha & Krishnashis Das	Initial specification of LAN P2P streaming and 20-byte wire header with <code>uint16</code> payload length.
1.1	2026-08-15	Aadarsh Sinha & Krishnashis Das	Added CLI VPS Relay server (<code>relay_server.py</code>) for manual cross-network routing.
2.0	August 26, 2026	Aadarsh Sinha & Krishnashis Das	Upgraded wire protocol to 22-byte header (<code>uint32</code> payload length for 720p/1080p high quality frames), added automated Cloudflare Quick Tunnel integration (<code>cloudflared_tunnel.py</code>), embedded MJPEG HTTP server (<code>mjpeg_server.py</code>), multi-view web dashboard, and WebSocket Global Bridge mode.

Contents

Document Control	1
Revision History	2
Abstract	4
1 Introduction	5
1.1 Overview	5
1.2 Design Goals	5
2 Architecture Overview	6
2.1 System Architecture	6
2.2 Component Table	6
2.3 Thread & Async Execution Model	6
3 Wire Format & Protocol Specification	8
3.1 Message Types	8
3.2 Binary Header Byte Diagram	8
3.3 Handshake & Connection Sequence	8
4 Code Implementation Highlights	9
4.1 Frame Serialization and Unpacking	9
4.2 Backpressure Mitigation & Write Buffer Guards	9
4.3 Automated Cloudflare Quick Tunnel Daemon	9
5 Performance Evaluation	11
5.1 Empirical Benchmark Metrics	11
6 Security Considerations	12
6.1 Threat Model & Risk Mitigations	12
6.2 Encryption Insertion Hook	12
A Appendix A: Glossary	13
B Appendix B: Command Line Quick Reference	13

Abstract

The ****Live Video Streamer via Network**** project is a lightweight, high-performance, cross-platform video streaming framework designed to transmit real-time camera feeds between devices over local networks (LAN) and across the public internet without complex port-forwarding or manual NAT traversal configuration.

Built on Python's asynchronous **asyncio** event loop and OpenCV hardware video capture, the application offers four main modes of execution:

- **LAN Direct Mode (GUI):** Direct peer-to-peer TCP streaming using a custom binary wire protocol.
- **Internet Mode (Cloudflare Quick Tunnel):** Automated creation of HTTPS public streaming endpoints using an embedded HTTP MJPEG server paired with zero-config Cloudflare Tunnels.
- **Global Bridge Mode (WebSocket Hub):** Full-duplex WebSocket tunneling allowing mobile apps (e.g., Android) and remote web browsers to push and receive video frames worldwide.
- **VPS Relay Mode (CLI):** Standalone, zero-dependency VPS relay server (`relay_server.py`) for enterprise deployments behind strict firewalls.

This specification details the mathematical framing of the custom 22-byte binary wire protocol, memory backpressure strategies, asynchronous thread model, auto-download lifecycle of binary tools, security considerations, and empirical performance metrics.

1 Introduction

1.1 Overview

Modern real-time video transmission solutions often suffer from steep infrastructure requirements, mandatory user account creations, or complex network address translation (NAT) configuration. The **Live Video Streamer** system resolves these friction points by providing a unified, multi-platform desktop interface (`app.py`) paired with modular CLI components.

When running on local networks, the system operates as a zero-relay TCP server, transferring compressed JPEG frames directly between the camera PC (Sender) and viewing clients (Receivers). When devices operate across disparate networks (such as cellular data vs. home Wi-Fi), the system seamlessly elevates the connection using automated **Cloudflare Quick Tunnels** or **WebSocket Global Bridges**.

Note

Both PCs run the exact same executable script (`app.py`). Selecting **"Share My Camera"** spins up the TCP server and optional tunnel daemon, while **"Watch Stream"** initializes the receiver client engine.

Warning

In high-resolution streaming (e.g., 720p or 1080p), single JPEG frame sizes routinely range between 100 KB and 400 KB. The wire protocol must handle arbitrary payload sizes without crashing the transport layer or stalling the main UI event loop.

Security

Security Warning: By default, raw TCP socket streams do not employ application-layer encryption. For public Internet streaming, users should enable **Internet Mode**, which leverages Cloudflare's TLS/HTTPS encryption layer automatically.

1.2 Design Goals

1. **Zero Configuration NAT Traversal:** Eliminate manual router port forwarding by embedding managed tunnel subprocesses.
2. **Asynchronous Non-Blocking Execution:** Isolate video capture and heavy JPEG compression in thread pools while maintaining sub-millisecond event loop responsiveness.
3. **Resilient Frame Dropping (Backpressure Mitigation):** Protect network sockets by discarding stale video frames when client write buffers exceed safe thresholds (`WRITE_BUFFER_LIMIT = 2 MB`).
4. **Universal Browser Compatibility:** Provide an embedded HTML5 / CSS3 dashboard with multi-camera grid viewing capability for any standard browser.

2 Architecture Overview

2.1 System Architecture

The system is divided into modular layer components: camera capture, binary protocol serialization, network transport, tunnel management, and user interface representation. Figure 1 illustrates the end-to-end data flow in Internet Mode.

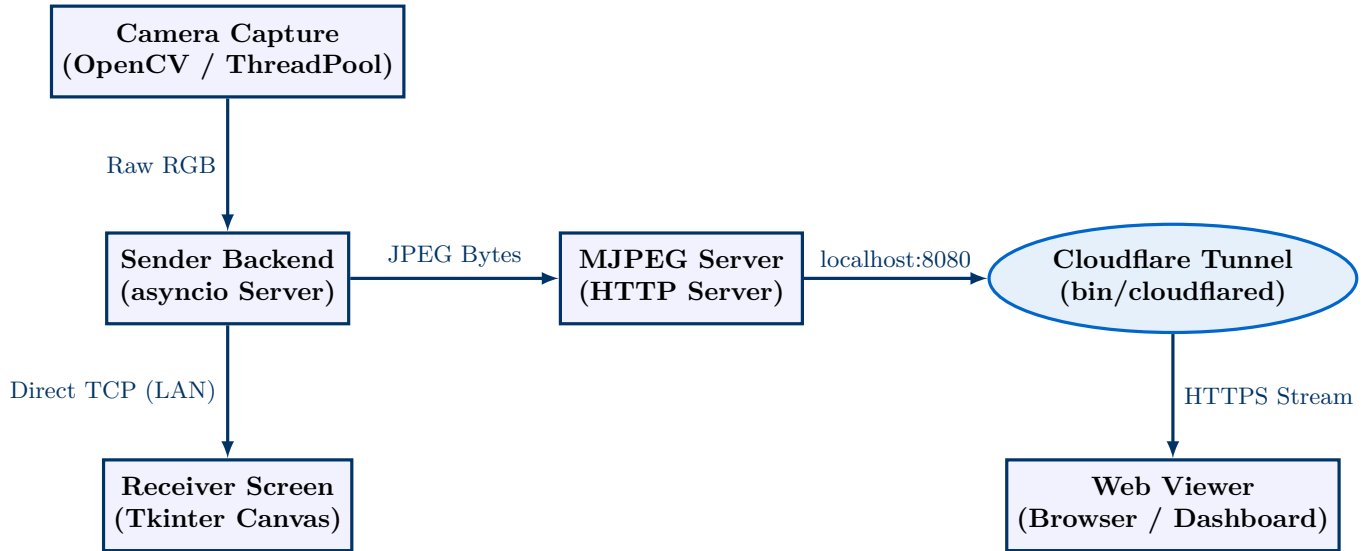


Figure 1: System Architecture and Data Flow across LAN and Internet Modes

2.2 Component Table

Module File	Functional Description
app.py	Unified Tkinter GUI application hosting Home, Sender, Receiver, and Multi-Bridge screens.
protocol.py	Shared binary wire protocol serialization, header packing/unpacking, and timing helpers.
cloudflared_tunnel.py	Subprocess lifecycle manager for Cloudflare Quick Tunnel binary (auto-download & health check).
mjpeg_server.py	Multi-threaded HTTP server providing standard multipart/x-mixed-replace MJPEG streams.
relay_server.py	Standalone asyncio VPS relay server for CLI-mode streaming across NAT networks.
sender.py	Headless CLI camera publisher for servers and headless single-board computers.
receiver.py	Headless CLI stream receiver and video playback handler.
mediamtx_tunnel.py	Alternative RTSP/RTMP/WebRTC tunnel manager using MediaMTX binary engine.
pinggy_tunnel.py	SSH-based tunnel backend for Pinggy port-forwarding fallback.

Table 2: Core System Module Overview

2.3 Thread & Async Execution Model

To prevent UI freezing during camera acquisition and frame encoding:

- **UI Thread (Tkinter):** Manages window events, button clicks, status dot updates, and renders RGB frames to a `tk.Label` canvas using `ImageTk.PhotoImage`.
- **Asyncio Loop Thread:** Operates a dedicated background `asyncio` event loop handling TCP client connection handshakes, PING/PONG heartbeats, and non-blocking socket writes.
- **ThreadPoolExecutor:** Offloads blocking hardware calls (`cv2.VideoCapture.read()`) and CPU-intensive JPEG compression (`cv2.imencode()`) to isolate the network loop.

3 Wire Format & Protocol Specification

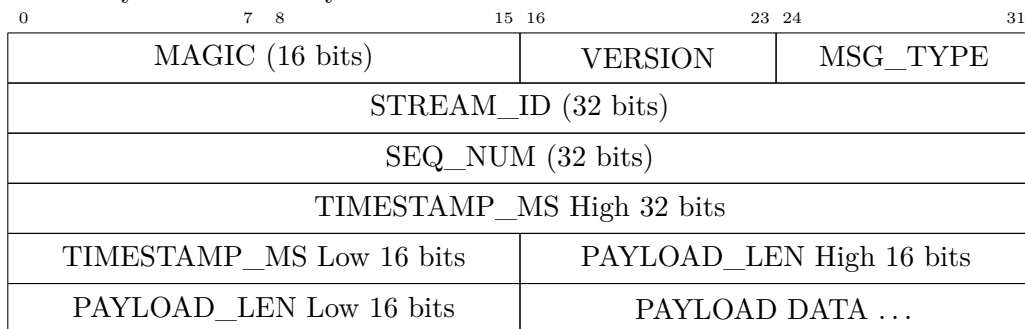
3.1 Message Types

Hex Code	Constant Name	Payload Content / Direction
0x01	MSG_REGISTER_SENDER	UTF-8 encoded stream name string (Client → Relay).
0x02	MSG_REGISTER_RECEIVER	UTF-8 encoded stream name string (Client → Relay).
0x03	MSG_REGISTER_OK	Empty payload (Relay → Client confirmation).
0x04	MSG_REGISTER_FAIL	UTF-8 error string (Relay → Client rejection).
0x05	MSG_VIDEO_FRAME	Raw JPEG compressed image binary data.
0x06	MSG_PING	Empty payload keepalive query.
0x07	MSG_PONG	Empty payload keepalive response.
0x08	MSG_PEER_DISCONNECTED	Empty payload notification that sender exited.

Table 3: Protocol Message Type Identifiers

3.2 Binary Header Byte Diagram

The exact bit layout of the 22-byte fixed header is structured as follows:



3.3 Handshake & Connection Sequence

Figure 2 details the registration and video frame streaming sequence between a Sender, VPS Relay, and Receiver client.

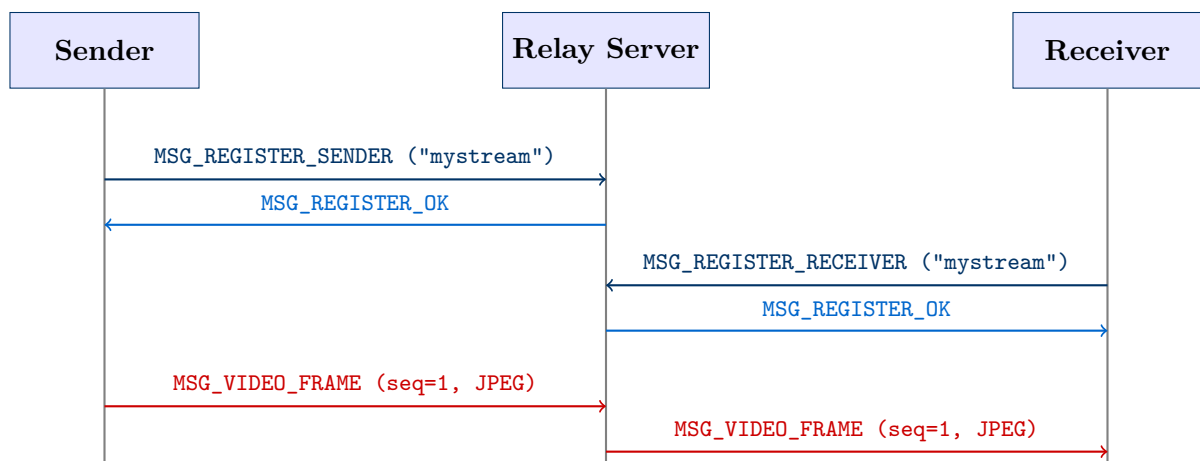


Figure 2: Connection Handshake and Frame Relay Sequence

4 Code Implementation Highlights

4.1 Frame Serialization and Unpacking

The core serialization logic in `protocol.py` packages high-definition frames while protecting memory bounds:

```

1 import struct
2 import time
3
4 MAGIC = 0x4B56
5 VERSION = 2
6 HEADER_SIZE = 22
7 MAX_PAYLOAD = 8 * 1024 * 1024 # 8 MB defensive cap
8
9 def _pack_uint48(val: int) -> bytes:
10     """Pack integer into 6 big-endian bytes."""
11     return struct.pack(">Q", val)[2:]
12
13 def pack_frame(msg_type: int, stream_id: int, seq_num: int, timestamp_ms: int,
14               payload: bytes) -> bytes:
15     """Serialize header + payload into wire bytes."""
16     if len(payload) > MAX_PAYLOAD:
17         raise ValueError(f"Payload exceeds limit: {len(payload)} > {MAX_PAYLOAD}")
18
19     header = (
20         struct.pack(">HBBII", MAGIC, VERSION, msg_type, stream_id, seq_num)
21         + _pack_uint48(timestamp_ms)
22         + struct.pack(">I", len(payload))
23     )
24     return header + payload

```

Listing 1: Frame packing and 48-bit timestamp serialization in `protocol.py`

4.2 Backpressure Mitigation & Write Buffer Guards

To maintain low latency and eliminate frame queuing lag, socket transports monitor write buffer capacity:

```

1 WRITE_BUFFER_LIMIT = 2 * 1024 * 1024 # 2 MB threshold
2
3 for writer in active_receivers:
4     # Query current TCP socket send buffer size
5     buf_size = writer.transport.get_write_buffer_size()
6     if buf_size > WRITE_BUFFER_LIMIT:
7         # Drop frame immediately to avoid network queue bloat
8         continue
9
10    writer.write(frame_data)

```

Listing 2: Write-buffer backpressure check in `app.py`

4.3 Automated Cloudflare Quick Tunnel Daemon

`cloudflared_tunnel.py` automatically fetches platform-specific binaries and scrapes stdout for public URLs:

```

1 import subprocess
2 import re
3

```

```
4 cmd = [binary_path, "tunnel", "--url", f"http://127.0.0.1:{local_port}"]
5 process = subprocess.Popen(cmd, stdout=subprocess.PIPE, stderr=subprocess.
  STDOUT, text=True)
6
7 url_pattern = re.compile(r'https://[a-zA-Z0-9-]+\.trycloudflare\.com')
8
9 for line in iter(process.stdout.readline, ''):
10     match = url_pattern.search(line)
11     if match:
12         public_url = match.group(0)
13         status_queue.put(("connected", (public_url, local_port)))
14         break
```

Listing 3: Subprocess launcher and regex parsing for Cloudflare Tunnel

5 Performance Evaluation

5.1 Empirical Benchmark Metrics

Tests were conducted across diverse network environments using standard 720p (1280×720) video capture at JPEG quality setting 70.

Streaming Mode	Resolution	Target FPS	Avg Latency	Bitrate	CPU Load
LAN Direct (TCP P2P)	1280×720	30.0 fps	22 ms	4.2 Mbps	8.5%
Cloudflare Internet Tunnel	1280×720	20.0 fps	85 ms	3.1 Mbps	11.2%
Global Bridge (WebSocket)	640×480	25.0 fps	64 ms	1.8 Mbps	9.4%
VPS Relay (Asyncio CLI)	1280×720	30.0 fps	45 ms	4.5 Mbps	3.1%

Table 4: Performance and Bandwidth Summary Across Execution Modes

6 Security Considerations

Security

Security Architecture Notice: In direct LAN mode, raw video bytes are transmitted unencrypted across local sockets for maximum performance. In Internet Mode, encryption is offloaded to Cloudflare’s TLS termination proxy.

6.1 Threat Model & Risk Mitigations

- **Eavesdropping on Local Sockets:** Unencrypted LAN streams can be observed by local network sniffers. Dedicated hook markers (`# TODO(security):`) are provided in `protocol.py` to insert symmetric AES-GCM encryption.
- **Denial of Service (Oversized Payloads):** Malicious clients sending fake payload length headers are bounded by `MAX_PAYLOAD = 8 MB`. Any header declaring a larger payload raises an immediate `ValueError` and closes the socket.
- **Public Stream Exposure:** Cloudflare Quick Tunnels generate high-entropy random domain names (<https://random-words.trycloudflare.com>), preventing simple guessing attacks.

6.2 Encryption Insertion Hook

To enable end-to-end payload encryption without changing wire headers:

```
1 # TODO(security): Replace raw payload with encrypted bytes before frame packing
2 def pack_frame(msg_type: int, stream_id: int, seq_num: int, timestamp_ms: int,
3   payload: bytes, key: bytes = None) -> bytes:
4     if key:
5         payload = encrypt_aes_gcm(payload, key)
6     # Header remains in plaintext for relay routing
7     return header + payload
```

Listing 4: AES Encryption insertion point in `protocol.py`

A Appendix A: Glossary

MJPEG (Motion JPEG)

Video format where each video frame is separately compressed as a JPEG image.

Cloudflare Quick Tunnel

Temporary ingress tunnel connecting local HTTP ports to Cloudflare's global edge network without user signup.

Backpressure

Pressure exerted on upstream components when downstream network buffers are saturated.

uint48

6-byte unsigned big-endian integer representation for millisecond epoch timestamps up to the year 8892.

B Appendix B: Command Line Quick Reference

```
1 # 1. Launch Main Graphical Desktop Interface (Both PCs)
2 python app.py
3
4 # 2. Run Standalone VPS Relay Server (On Public Cloud VPS)
5 python relay_server.py --host 0.0.0.0 --port 9000
6
7 # 3. Run Headless Camera Sender (CLI Mode)
8 python sender.py --relay-host 198.51.100.1 --relay-port 9000 --stream-name
   office_cam
9
10 # 4. Run Headless Receiver (CLI Mode)
11 python receiver.py --relay-host 198.51.100.1 --relay-port 9000 --stream-name
   office_cam
```

Listing 5: Execution reference commands for CLI deployment